

Dependence Comparison of the Effectiveness of Software Tools for Splitting Video into Frames on Format and Resolution Using a Road Survey as an Example

Alexander A. Zhuravlev
Ural Federal University n. a. the first
President of Russia B.N. Yeltsin
Yekaterinburg, Russia
alexander.zhuravlev@urfu.ru

Konstantin A. Aksyonov
Ural Federal University n. a. the first
President of Russia B.N. Yeltsin
Yekaterinburg, Russia
k.a.aksenov@urfu.ru

Abstract—The importance of roads in the life of almost every modern person is quite huge. One of the key characteristics of the road surface is its quality. To assess the quality of roads, there are many systems, most of which work with a stream of video information. This problem is solved mainly by analyzing images that are obtained from the video stream. To split video into frames, there are special software tools with varying effectiveness. In this research, the average time to retrieve one frame from a video is selected as the main performance indicator. FFMpeg, MoviePy and OpenCV are selected as analyzed programs for splitting videos into frames. The study will help determine the most effective software tool and format for postprocessing (splitting into frames) videos received from any source: a phone camera, drone, etc. The research material is a minute-long video containing a road surface (any other video can be used, it is just example) with a frequency of 25 frames/sec. in 4 formats (avi, mov, mp4, wmv) and 5 resolutions of 16:9 (240p, 320p, 480p, 720p, 1080p) – a total of 20 different videos for each of the selected software tools (total number of experiments are 60). All selected programs have linear dependence on the resolution for each format. However, FFMpeg software tool and wmv format have the lowest absolute time values; FFMpeg also has a significantly better efficiency compared to MoviePy and OpenCV.

Keywords—video splitting, software tool, effectiveness, dependence comparison, OpenCV, MoviePy, FFMpeg, frame, video format, resolution, road survey

I. INTRODUCTION

The problem of the road quality assessment is one of the most popular all over the world. Quite many scientific papers have been written on this topic. The works are mainly devoted to the processing of video and images of the road surface using various methods. They can be divided into two classes: traditional (without using machine learning) [1-4] and using matching learning [5-9]. Currently, machine learning approaches are the most relevant, due to the popularity of neural networks. Some works are devoted to evaluating the effectiveness of various methods [10-13]. This study will focus specifically on effectiveness. Video splitting into frames by various software tools (libraries) such as FFMpeg, MoviePy and OpenCV is chosen as area research. Each of the selected programs may have certain features, such as the complexity of processing video of a certain resolution, which, in turn, can have a significant impact on the dependence type. For example, it can be quadratic - $O(n^2)$, or cubic - $O(n^3)$. Also, software tools, if they have the same dependence (for example, linear), may have different growth rates of the function, or have significantly different

absolute time values. The quadratic or cubic function type, high growth rate and large absolute time values have a significant impact on the system performance.

To understand what characteristics, the selected software tools have, it is necessary to conduct an appropriate experiment.

The **purpose** of the work is to compare the efficiency of software tools for splitting video into frames on format and resolution.

The **research material** is a minute-long video of various formats and resolutions of 16:9.

The **scientific novelty** lies in the approach of determining the effectiveness of software tools for splitting video into frames.

The research objectives:

1. Briefly describe software tools for splitting video into frames selected for the experiment.
2. To describe the methodology of the experiment.
3. To demonstrate and describe the results obtained.

II. SOFTWARE TOOLS FOR VIDEO SPLITTING

OpenCV, MoviePy, FFMpeg are selected as software tools for analysis. These programs were chosen because they can be experimented with in a single programming environment. Brief information about each tool (library) is provided below.

A. FFMpeg

FFMpeg is a universal software tool that allows to read a wide variety of input data, including real-time capture/recording devices, filter and encoding into a variety of output formats [14, 15]. Requires pre-installation on a computer with a Windows operating system.

B. MoviePy

MoviePy is a Python library for the video editing: cropping, title insertion, processing and creating custom effects. This tool can read and write all the most common audio and video formats, including GIF, and runs on Windows/Max/Linux with Python 2.7+ and 3 [16, 17].

C. OpenCV

OpenCV is a free cross-platform library for real-time video and image processing. This software tool runs on the most popular operating systems such as Windows, OS X, GNU/Linux, iOS, Android, etc. The OpenCV library contains more than 2500 algorithms, documentation, source code and code examples for real-time computer vision [18, 19].

III. METHODOLOGY

The videos of the road surface are used in the experiment (any other videos can be used). They have the following characteristics:

- Length – 1 minute.
- Frame rate – 25 frames/sec.
- Formats – avi, mov, mp4, wmv.
- Resolutions – 240p (426 x 240 = 102240 pixels), 360p (640 x 360 = 230400), 480p (854 x 480 = 409920), 720p (1280 x 720 = 921 600), 1080p (1920 x 1080 = 2073600).

The full characteristics of the experimental video (using 240p as an example) are shown in Fig. 1.

Video

Length	00:01:00
Frame width	426
Frame height	240
Data rate	728kbps
Total bitrate	856kbps
Frame rate	25.00 frames/second

Fig. 1. The full characteristics of the experimental video (using 240p as an example)

The experimental computer has the following characteristics: RAM – 8 GB, RAM type – DDR4, RAM channel – single, processor – Intel Core i5, processor frequency – 1.6 gigahertz, operating system – Windows 10 Pro, the system type – x64, hard drive type – basic.

Python (version 3.11.7) [20] is chosen as the programming language, the application type is Jupyter-notebook [21], Visual Studio Code 2023 [22] is a development environment. CPU is used for the experiment.

Description of an experiment to determine the dependence of the average time to retrieve one frame from a video on the format and resolution:

1. The videos described above are used as research material for each of the selected software tools (FFMpeg, MoviePy, OpenCV).

2. A series of 20 experiments is conducted for each program, in which 5 different resolutions of 16:9 (240p, 360p, 480p, 720p, 1080p) are used for each of the 4 selected formats (avi, mov, mp4, wmv). The video for each experiment is stored in a separate computer folder (a total of 60 different folders: 3 (software tools) x 4 (formats) x 5 (resolutions)). All frames are only saved (without output to the screen) to the same folder where the video is located.

3. To determine the average time to retrieve one frame from a video, the total time spent splitting the entire video into frames is divided by the total number of frames (1).

$$t_{avr} = \frac{t_{total}}{N} \quad (1)$$

where t_{avr} is the average time to retrieve one frame from the video, t_{total} is the total time to split the video into frames, N is the number of frames.

The stages of conducting experiments to calculate the average time to retrieve one frame from a video for a software tool are presented in the block diagram demonstrated in Fig. 2. Time is fixed in Python using the time function of the time module. It outputs a value with 7 digits after the dot [23].

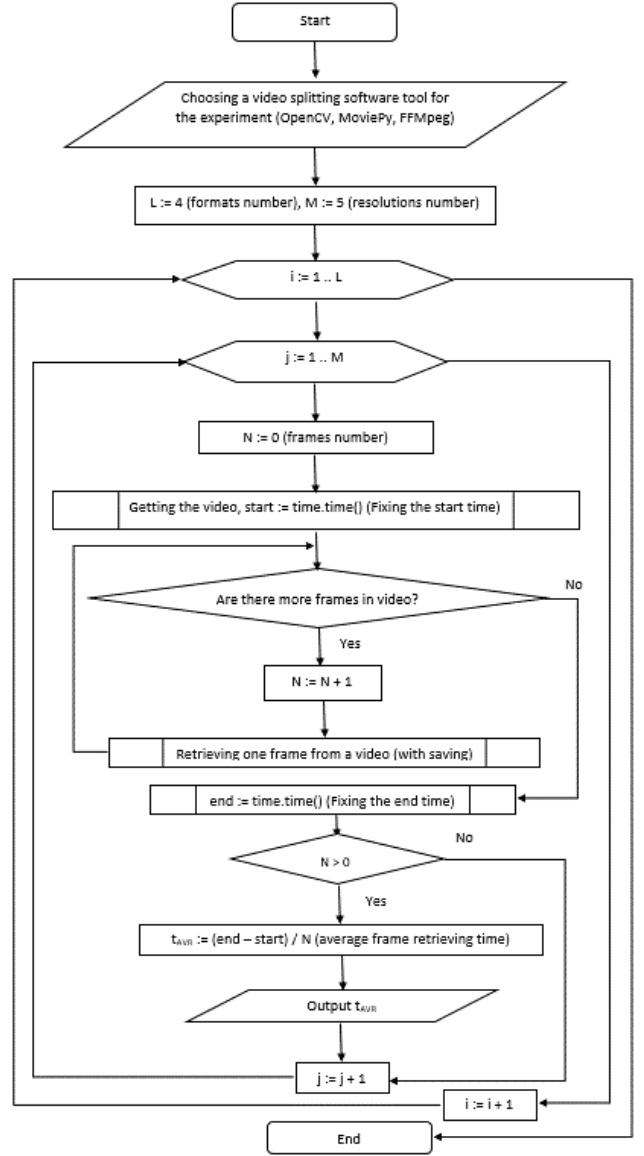


Fig. 2. The block diagram of the algorithm for calculating the average time to retrieve one frame from a video using a software tool for different formats and resolutions

IV. RESULTS AND DISCUSSION

After conducting the experiments described earlier, a total of 1503 frames are obtained for each video (60 sec. (minute in seconds) x 25 frames/sec. $\approx$ 1503 frames). The value turns out to be slightly large, since the video length is rounded by the computer.

An example of an avi-format video and its retrieved frames is shown in Fig. 3.



Fig. 3. An example of an avi-format video and its retrieved frames

In Fig. 3 “...” are frames from 4 to 1499.

The results of obtaining the total time of video splitting for the selected formats and resolutions, as well as the average time for retrieving one frame (dividing the total time by the number of frames), are presented in Tables I and II, respectively (for Table II the values are rounded to 3 decimal places).

TABLE I. THE TOTAL TIME (IN MILLISECONDS) OF SPLITTING ONE-MINUTE-LONG VIDEO WITH 25 FRAMES/SEC. FRAME RATE BY SELECTED SOFTWARE TOOLS DEPENDING ON THE FORMAT AND RESOLUTION

AVI					
Format	Resolution				
	240p	360p	480p	720p	1080p
FFMpeg	1825	3330	5150	8916	17894
MoviePy	3667	6202	9283	18201	39371
OpenCV	6683	13100	21334	42767	92470
MOV					
Format	Resolution				
	240p	360p	480p	720p	1080p
FFMpeg	1905	3312	5127	8994	17985
MoviePy	3861	6179	9229	17917	41097
OpenCV	6842	12824	20848	43649	91879
MP4					
Format	Resolution				
	240p	360p	480p	720p	1080p
FFMpeg	1775	3341	5582	9232	18376
MoviePy	4145	6471	9232	16634	40239
OpenCV	7637	13099	20773	42424	91765
WMV					
Format	Resolution				
	240p	360p	480p	720p	1080p
FFMpeg	1608	2487	4120	7990	16196
MoviePy	3302	5911	7709	14514	32543
OpenCV	5674	10333	17342	33882	74760

TABLE II. THE AVERAGE TIME (IN MILLISECONDS) OF RETRIEVING ONE FRAME FROM A VIDEO BY SELECTED SOFTWARE TOOLS DEPENDING ON THE FORMAT AND RESOLUTION

AVI					
Format	Resolution				
	240p	360p	480p	720p	1080p
FFMpeg	1.214	2.216	3.426	5.932	11.906
MoviePy	2.440	4.126	6.176	12.110	26.195
OpenCV	4.447	8.716	14.194	28.455	61.524
MOV					
Format	Resolution				
	240p	360p	480p	720p	1080p
FFMpeg	1.267	2.204	3.411	5.984	11.966
MoviePy	2.569	4.111	6.140	11.921	27.343
OpenCV	4.552	8.532	13.871	29.041	61.131
MP4					
Format	Resolution				
	240p	360p	480p	720p	1080p
FFMpeg	1.181	2.218	3.714	6.143	12.226
MoviePy	2.758	4.305	6.143	11.067	26.773
OpenCV	5.081	8.715	13.821	28.226	61.055
WMV					
Format	Resolution				
	240p	360p	480p	720p	1080p
FFMpeg	1.070	1.654	2.741	5.316	10.776
MoviePy	2.222	3.933	5.129	9.656	21.652
OpenCV	3.775	6.875	11.538	23.208	49.741

To make the results clearer, the dependence graphics of the video format on the software tools and resolutions (240p, 360p, 480p, 720p, 1080p) for Table I and Table II were plotted. The graphics are demonstrated in Fig. 4 and Fig. 5.

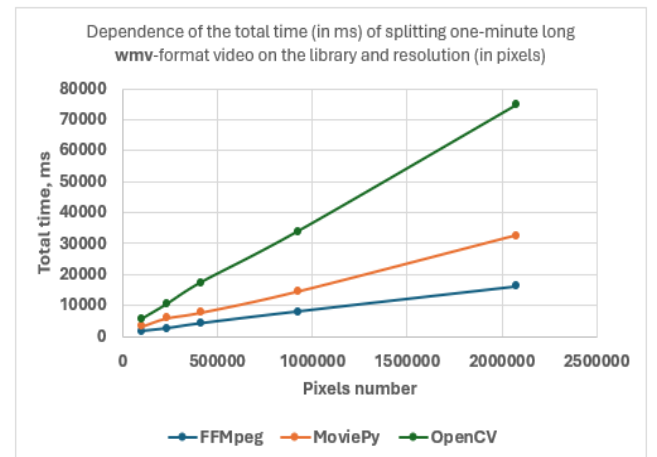
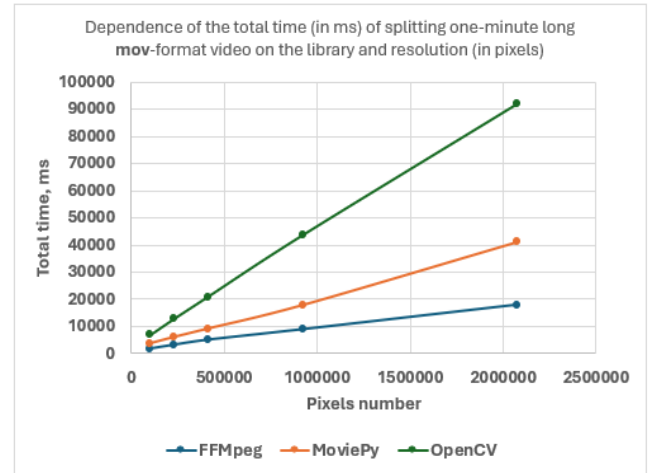
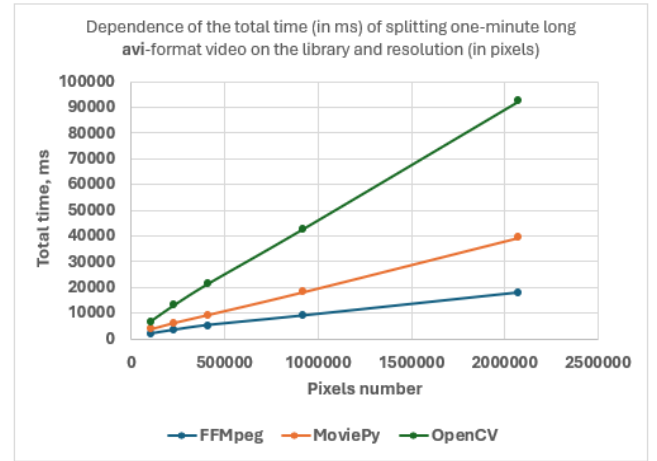


Fig. 4. The dependence of the total time of splitting different format one-minute-long videos with 25 frames/sec. frame rate on software tools and resolutions

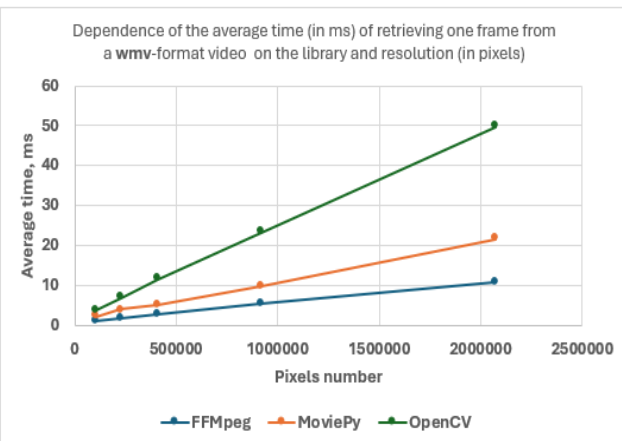
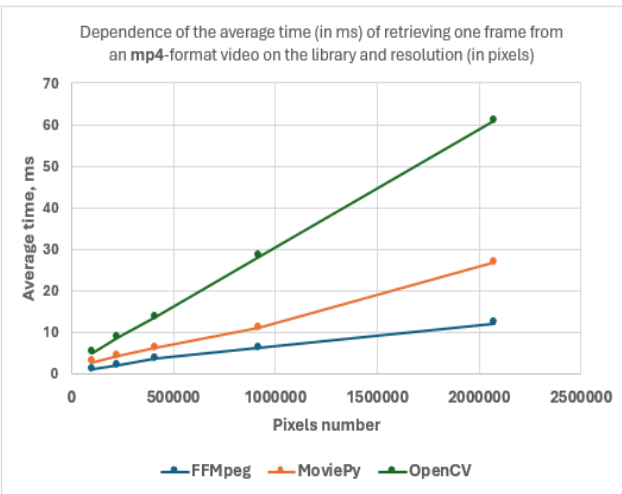
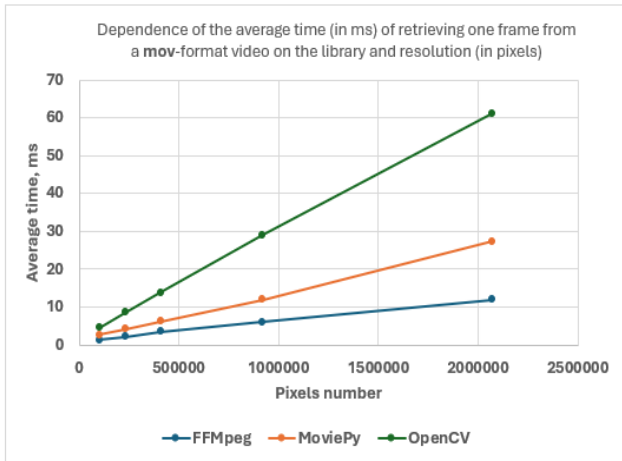
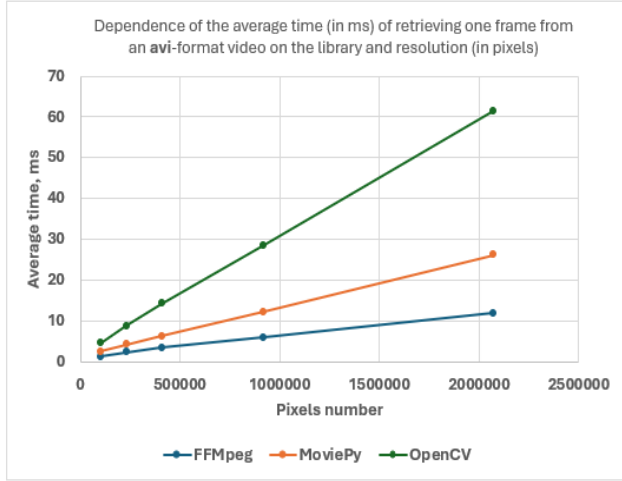


Fig. 5. The dependence of the average time to retrieve one frame from different format videos on software tools and resolutions

As can be seen, all graphics have a linear dependence $O(n)$ [24, 25]. However, FFMpeg software tool and wmv format have the lowest absolute time values, FFMpeg also has a significantly lower function growth rate compared to MoviePy and OpenCV.

Let's calculate the efficiency depending on the software tool function change rate for each of the formats in such a way that the sum of the efficiencies of software tools (FFMpeg, MoviePy, OpenCV) gives 1 for each format. For the sum of the efficiencies to give 1, a normalization coefficient is needed. Efficiency is inversely proportional to the rate of growth of the function. For a clearer representation of the effectiveness of the software tool for all formats, let's also calculate the average value. The following formulas are needed:

$$v_{avr} = \frac{\Delta t}{\Delta x} = \frac{t_{max} - t_{min}}{x_{max} - x_{min}} \quad (2)$$

where v_{avr} is the average function change rate, Δt is the difference between the maximum (t_{max}) and minimum (t_{min}) absolute time values (total or average), Δx is the difference between the maximum (x_{max}) and minimum (x_{min}) resolution values.

$$k_{norm} * \left(\sum_{i=1}^S \frac{1}{v_{avr i}} \right) = k_{norm} * \left(\sum_{i=1}^S \frac{\Delta x_i}{\Delta t_i} \right) = 1 \quad (3)$$

Let's express k_{norm} from formula 3:

$$k_{norm} = \frac{1}{\sum_{i=1}^S \frac{\Delta x_i}{\Delta t_i}} \quad (4)$$

where k_{norm} is the normalization coefficient, $v_{avr i}$ is the function change rate of a particular software tool, S is the number of software tools (in our case 3).

$$E = \frac{k_{norm}}{v_{avr}} = \frac{k_{norm} * \Delta x}{\Delta t} \quad (5)$$

where E is the effectiveness of a particular software tool.

$$E_{avr} = \frac{\sum_{i=1}^L E_i}{L} \quad (6)$$

where E_{avr} is the average efficiency of the software tool, E_i is the efficiency of the library for a certain format, L is the number of formats (equal to 4).

The software tools (FFMpeg, MoviePy, OpenCV) efficiencies for each video format (avi, mov, mp4, wmv) are presented in Table III (the values are rounded to 3 decimal places).

TABLE III. THE SOFTWARE TOOLS EFFICIENCIES FOR EACH VIDEO FORMATS

Format	Software tool		
	FFMpeg	MoviePy	OpenCV
avi	0.611	0.275	0.114
mov	0.617	0.266	0.117
mp4	0.603	0.278	0.119
wmv	0.585	0.292	0.123
Average value	0.604	0.278	0.118

As can be seen in the table, FFMpeg has significantly better average effectiveness (0.604) compared to MoviePy (0.278) and OpenCV (0.118).

For a clearer representation of the results obtained in Table III, the effectiveness diagrams for all video formats and their average values are plotted. Diagrams are presented in Fig. 6.

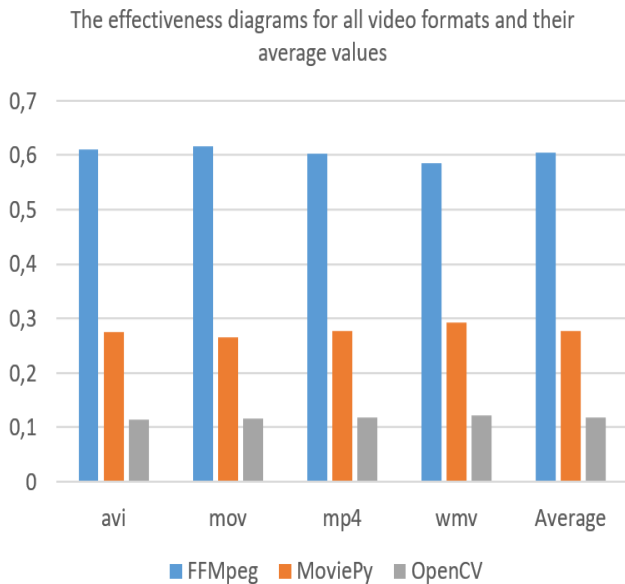


Fig. 6. The effectiveness diagrams for all video formats and their average values

In [13], the efficiencies of MoviePy and OpenCV are also compared, but FFMpeg is not considered. Also, an additional advantage of this research is considering the performance of software tools for multiple video resolutions. And since each of the considered programs has a linear dependence on time, the approximate time to retrieve a frame from a video can be calculated for any resolution.

The images obtained during the experiment can be further used as a set for training neural networks to detect various road damages with subsequent assessment of its quality.

V. CONCLUSION

During this study, a comparative analysis of the efficiency dependence of software tools for splitting video into frames on formats (avi, mov, mp4, wmv) and resolutions (240p, 360p, 480p, 720p, 1080p) was carried out using the example of the field of road surface quality assessment. The research material is minute-long videos of the road surface with frames rate of 25 frames/sec. of various formats and resolutions. FFMpeg, MoviePy and OpenCV are selected as programs (libraries) for analysis. A series of 20 experiments was conducted for each of the selected libraries, in which 5 different 16:9 resolutions (240p, 360p, 480p, 720p, 1080p) are used for each of the 4 selected formats (avi, mov, mp5, wmv). The average time to retrieve one frame from a video is selected as the main performance indicator. For the results obtained, the dependence graphics of the video format on the software tools and resolutions were plotted. Each of the graphics has a linear dependence $O(n)$. However, FFMpeg software tool and wmv format have the lowest absolute time values; FFMpeg also has a significantly better efficiency compared to MoviePy and OpenCV.

REFERENCES

- [1] R. Fan and M. Liu, "Road Damage Detection Based on Unsupervised Disparity Map Segmentation", Institute of Electrical and Electronics Engineers Inc., Vol. 2, 2019, pp. 4906-4911.
- [2] A. Danti, J. Y. Kulkarni, and P. S. Hiremath, "An Image Processing Approach to Detect Lanes, Pot Holes and Recognize Road Signs in Indian Roads", International Journal of Modeling and Optimization, Vol. 2, No. 6, 2012, pp. 658-662.
- [3] R. Sulistyowati, A. Suryowinoto, H. A. Sujono, and I. Iswahyudi, "Monitoring of road damage detection systems using image processing methods and Google Map", IOP Conference Series: Materials Science and Engineering, Vol. 1010, 2021, article 012017.
- [4] P. Liang, W. Shi, Y. Ding, Z. Liu, and H. Shang, "Road Extraction from High Resolution Remote Sensing Images Based on Vector Field Learning", Sensors, Vol. 21, No. 9, 2021, pp. 1-16.
- [5] S. Naddaf-Sh, M-M. Naddaf-Sh, A. R. Kashani, and H. Zargarzadeh, "An Efficient and Scalable Deep Learning Approach for Road Damage Detection", Institute of Electrical and Electronics Engineers Inc., in 2020 IEEE International Conference on Big Data (Big Data), 2020, pp. 5602-5608.
- [6] L. A. Silva, V. R. Q. Leithardt, V. F. L. Batista, G. V. González, and J. F. D. P. Santana, "Automated Road Damage Detection Using UAV Images and Deep Learning Techniques", Institute of Electrical and Electronics Engineers Inc., IEEE Access, Vol. 11, 2023, pp. 62918-62931.
- [7] K. C. Rehana, G. Remya, "Road damage detection algorithm for improved YOLOv5", Institute of Electrical and Electronics Engineers Inc., in 2022 Third International Conference on Intelligent Computing Instrumentation and Control Technologies (ICICICT), 2022, article 9917763.
- [8] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, A. Mraz, T. Kashiya, and Y. Sekimoto, "Deep learning-based road damage detection and classification for multiple countries // Automation in Construction", Vol. 132, 2021, article 103935.
- [9] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiya, and H. Omata, "Road Damage Detection and Classification Using Deep Neural Networks with Smartphone Images: Road damage detection and classification" // Computer-Aided Civil and Infrastructure Engineering, Vol. 33, No. 12, 2018, pp. 1127-1141.
- [10] A. A. Zhuravlev, "COMPARISON OF THE CLASSIFICATION EFFICIENCY OF THE CONTOUR DETECTION METHODS ON THE EXAMPLE OF ROAD SURFACE IMAGES", XXI century: results of the past and problems of the present plus, Vol. 12, No. 1, 2023, pp 23-28.
- [11] A. A. Zhuravlev, K. A. Aksyonov, "Comparison of Contour Detection Methods in Images on the Example of Photos with Road Surface Damage", Institute of Electrical and Electronics Engineers Inc., in 2023 IEEE Ural-Siberian Conference on Biomedical Engineering, Radioelectronics and Information Technology (USBREIT), 2023, pp. 183-186.
- [12] A. A. Zhuravlev, "Improving images of asphalt concrete pavements based on segmentation methods", Inzhenernyj vestnik Dona, No. 7, 2023, pp. 393-403.
- [13] A. Saudagar, H. Mohammed, "A comparative Study of Video Splitting Techniques", Progress in Systems Engineering, in Proceedings of the 23rd International Conference on Systems Engineering, Vol. 366, 2014, pp. 783-788.
- [14] "FFMpeg Documentation", <https://ffmpeg.org/ffmpeg.html#:~:text=ffmpeg%20is%20a%20universal%20media,a%20plethora%20of%20output%20formats>.
- [15] Tim Nugent, "A quick guide to using Ffmpeg to convert media files", <https://opensource.com/article/17/6/ffmpeg-convert-media-file-formats>
- [16] "MoviePy", <https://pypi.org/project/moviepy/>
- [17] "Introduction to MoviePy", <https://www.geeksforgeeks.org/introduction-to-moviepy/>
- [18] "What is OpenCV? The Complete Guide (2024)", <https://viso.ai/computer-vision/opencv/>
- [19] "OpenCV-Python Tutorials", https://docs.opencv.org/3.4/d6/d00/tutorial_py_root.html
- [20] "Python 3.11.7 documentation", <https://docs.python.org/release/3.11.7/>
- [21] "Project Jupyter Documentation", <https://docs.jupyter.org/en/latest/>
- [22] "Documentation for Visual Studio Code", <https://code.visualstudio.com/docs>
- [23] "time — Time access and conversions", <https://docs.python.org/3/library/time.html>
- [24] "Analysis of Algorithms | Big-O analysis", <https://www.geeksforgeeks.org/analysis-algorithms-big-o-analysis/>
- [25] R. Rasooli, "Time Complexity of Algorithms", <https://rpubs.com/rooholamin/big-o>